



ESCUELA DE INGENIERÍA DE
FUENLABRADA

GRADO EN
INGENIERÍA DE ROBÓTICA SOFTWARE

TRABAJO FIN DE GRADO

IMPLEMENTACIÓN DE UN ROBOT POR CABLES PARA EL
CONTROL DE UN EFECTOR FINAL EN DIVERSAS TAREAS

Autor: Alberto León Luengo

Tutor: Dr. Juan Sebastián Cely Gutiérrez

Curso académico 2025-2026



Este trabajo se distribuye bajo los términos de la licencia internacional CC BY-SA International License (Creative Commons Attribution-ShareAlike 4.0). Usted es libre de *(a) compartir*: copiar y redistribuir el material en cualquier medio o formato para cualquier propósito, incluso comercialmente; y *(b) adaptar*: remezclar, transformar y construir a partir del material para cualquier propósito, incluso comercialmente. La licenciente no puede revocar estas libertades en tanto usted siga los términos de la licencia:

- *Atribución.* Usted debe dar crédito de manera adecuada, brindar un enlace a la licencia, e indicar si se han realizado cambios. Puede hacerlo en cualquier forma razonable, pero no de forma tal que sugiera que usted o su uso tienen el apoyo de la licenciente.
- *Compartir igual.* Si remezcla, transforma o crea a partir del material, debe distribuir su contribución bajo la misma licencia del original.

No hay restricciones adicionales — No puede aplicar términos legales ni medidas tecnológicas que restrinjan legalmente a otras a hacer cualquier uso permitido por la licencia.

Agradecimientos

Me gustaría mostrarles todo mi agradecimiento a mis padres, Natividad y Alberto, que han estado siempre apoyándome en todo momento y ayudándome a superar los obstáculos más difíciles a los que he tenido que hacer frente a lo largo de estos seis años de etapa universitaria.

A mi hermana Ainhoa, por ser capaz de sacarme una sonrisa siempre que la situación no ha sido favorable y por ayudarme a salir de mi rutina habitual manteniéndome dentro de mi zona de confort.

A Juan, mi tutor, por el buen trato y todos los nuevos conocimientos y enseñanzas que he podido aprender gracias a él en todo el tiempo que he compartido con él realizando este TFG.

Y por último, a todos los compañeros que me han estado acompañando en la recta final de este viaje, y que gracias al apoyo que ellos me han ofrecido y que yo de la misma forma les he ofrecido a ellos, todos mis objetivos y éxitos esperados han acabado dando sus frutos.

Acrónimos

TFG *Trabajo de Fin de Grado*

CDPR *Cable Driven Parallel Robot*

ROS *Robot Operating System*

RViZ *ROS Visualization*

URDF *Unified Robot Description Format*

Índice general

1. Introducción	1
1.1. Contexto y motivación	1
1.2. Planteamiento del problema	3
1.3. Objetivos	4
1.4. Alcance y limitaciones	5
1.5. Metodología	6
1.5.1. Fase 1	6
1.5.2. Fase 2	6
1.5.3. Fase 3	6
1.5.4. Fase 4	6
1.6. Descripción inicial del sistema	6
1.7. Herramientas de apoyo empleadas durante el desarrollo	7
1.8. Relación con los conocimientos adquiridos en el grado	8
2. Fundamentos teóricos	11
2.1. Introducción	11
2.2. Robots paralelos accionados por cables (CDPR)	11
2.3. Configuraciones típicas	11
2.4. Modelo cinemático	12
2.5. Control y planificación de trayectorias	12
2.6. Frameworks robóticos	12
2.7. Aplicaciones y líneas de investigación actuales	13
2.8. Relación con el trabajo de fin de grado desarrollado	13
3. Plataforma de desarrollo	14
3.1. Introducción	14
3.2. Funcionamiento de un CDPR bidimensional	14
3.3. Arquitectura	14
3.4. Componentes software	16

3.5. Sistema hardware	16
3.6. Flujo general de funcionamiento	17
3.7. Relación entre módulos	19
4. Modelo cinemático	20
4.1. Descripción geométrica del sistema	21
4.2. Definición de poleas y parámetros conocidos	22
4.3. Geometría del efector final	23
4.4. Cinemática inversa	24
4.5. Cálculo de la orientación de los cables	25
4.6. Relación entre cables y poleas	26
4.7. Espacio de trabajo y restricciones geométricas	26
4.8. Consideraciones y limitaciones del modelo	27
5. Simulación y validación del modelo	28
5.1. Introducción	28
5.2. Simulación inicial en MATLAB	28
5.3. Implementación del modelo en Python	29
5.4. Implementación del modelo en C++	31
5.5. Comparación y validación de resultados	31
5.6. Análisis de errores y limitaciones	32
6. Integración del sistema en ROS 2	33
6.1. Introducción	33
6.2. Estructura del paquete ROS 2	33
6.3. Nodo de control del CDPR	34
6.4. Comunicación mediante topics	36
6.5. Gestión de trayectorias y movimientos	38
6.6. Visualización del sistema en RViZ	38
6.7. Compilación y ejecución del sistema	39
7. Desarrollo hardware	40
7.1. Introducción	40
7.2. Diseño mecánico del sistema	40
7.3. Sistema electrónico	40
7.4. Programación del microcontrolador	40
7.5. Integración hardware-software	40

7.6. Consideraciones prácticas y limitaciones del prototipo	40
7.7. Resumen	40
8. Pruebas experimentales y validación	41
8.1. Introducción	41
8.2. Metodología de pruebas	41
8.3. Pruebas de posicionamiento del efector final	41
8.4. Prueba de trayectorias	41
8.5. Comparación entre simulación y resultados	41
8.6. Análisis de errores y fuentes de incertidumbre	41
8.7. Resumen	41
9. Discusión de resultados	42
9.1. Introducción	42
9.2. Evaluación del cumplimiento de los objetivos	42
9.3. Análisis de los resultados de la simulación	42
9.4. Análisis de los resultados experimentales	42
9.5. Comparación entre comportamiento teórico y real	42
9.6. Limitaciones del sistema desarrollado	42
9.7. Propuesta de mejora y líneas de evolución	42
9.8. Resumen	43
10. Conclusiones y mejoras a futuro	44
10.1. Introducción	44
10.2. Conclusiones generales	44
10.3. Grado de cumplimiento de los objetivos	44
10.4. Aportaciones del trabajo	44
10.5. Limitaciones finales del sistema	44
10.6. Mejoras a futuro	44
10.7. Cierre del trabajo	44
Bibliografía	45

Índice de figuras

1.1. Ejemplo de CDPR de dos grados de libertad. En este tipo de sistemas, el movimiento del efector final se obtiene mediante la variación de la longitud de los cables conectados a poleas fijas de la estructura [1].	2
1.2. Esquema conceptual del problema abordado	3
1.3. Descripción general de los componentes del CDPR	7
3.1. Arquitectura general del sistema en ROS 2	15
3.2. Flujo general de funcionamiento del sistema	18
4.1. Modelo cinemático de un CDPR bidimensional	21
4.2. Geometría y dimensiones del efector final	23
5.1. Simulación inicial en MATLAB	29
5.2. Implementación y simulación en Python	30
5.3. Implementación y simulación en C++	31
6.1. Estructura del nodo cdpr_controller	35
6.2. Visualización en RViZ durante la ejecución de una trayectoria	36
6.3. Comunicación mediante topics en ROS 2	37

Listado de ecuaciones

4.1. Posición del efector final en el espacio de trabajo	22
4.2. Posición de la polea izquierda en el espacio de trabajo	22
4.3. Posición de la polea derecha en el espacio de trabajo	22
4.4. Posición de la esquina superior izquierda del efector final	23
4.5. Posición de la esquina superior derecha del efector final	23
4.6. Posición de la esquina inferior izquierda del efector final	24
4.7. Posición de la esquina inferior derecha del efector final	24
4.8. Longitud del cable izquierdo	24
4.9. Longitud del cable derecho	25
4.10. Ángulo que forma el cable izquierdo con la vertical	25
4.11. Ángulo que forma el cable derecho con la vertical	25
4.12. Relación longitud cable - radio polea	26

Índice de tablas

1.1. Relación entre asignaturas del grado y su aportación al TFG	9
--	---

Capítulo 1

Introducción

1.1. Contexto y motivación

La robótica ha experimentado un crecimiento significativo en las últimas décadas, ampliando su presencia en gran variedad de sectores como la industria, logística, medicina, construcción y entretenimiento, entre otros. Es por ello que, el desarrollo de nuevas arquitecturas robóticas capaces de maniobrar en grandes espacios de trabajo y estructuras ligeras, se encuentran dentro de una amplia línea de investigación tanto para la comunidad científica como para la industrial.

Los robots paralelos accionados por cables, también conocidos como Cable Driven Parallel Robots (CDPR), han despertado bastante interés en los últimos años tanto en el ámbito académico como en el ámbito industrial. Este tipo de robots basa su funcionamiento en el uso de cables que pueden ser recogidos o elongados por poleas con el objetivo de desplazar el efector final que une todos los cables a una posición específica, obteniendo como resultado una estructura ligera, escalable, y lo más importante de todo, capaz de realizar grandes volúmenes de trabajo en comparación con otros tipos de robots.

Este tipo de robots representan una alternativa bastante atractiva en comparación con otras arquitecturas robóticas debido a su capacidad para cubrir grandes espacios de trabajo utilizando mecanismos sencillos y ligeros. Además, para hacerlos funcionar, se requiere un modelo matemático del sistema que permita planificar trayectorias y que pueda integrarse en un entorno de desarrollo software, que puede llegar a convertirse en una plataforma que aborde problemas cinemáticos, simulación e integración basados en una arquitectura utilizada en robótica, como lo es ROS 2.

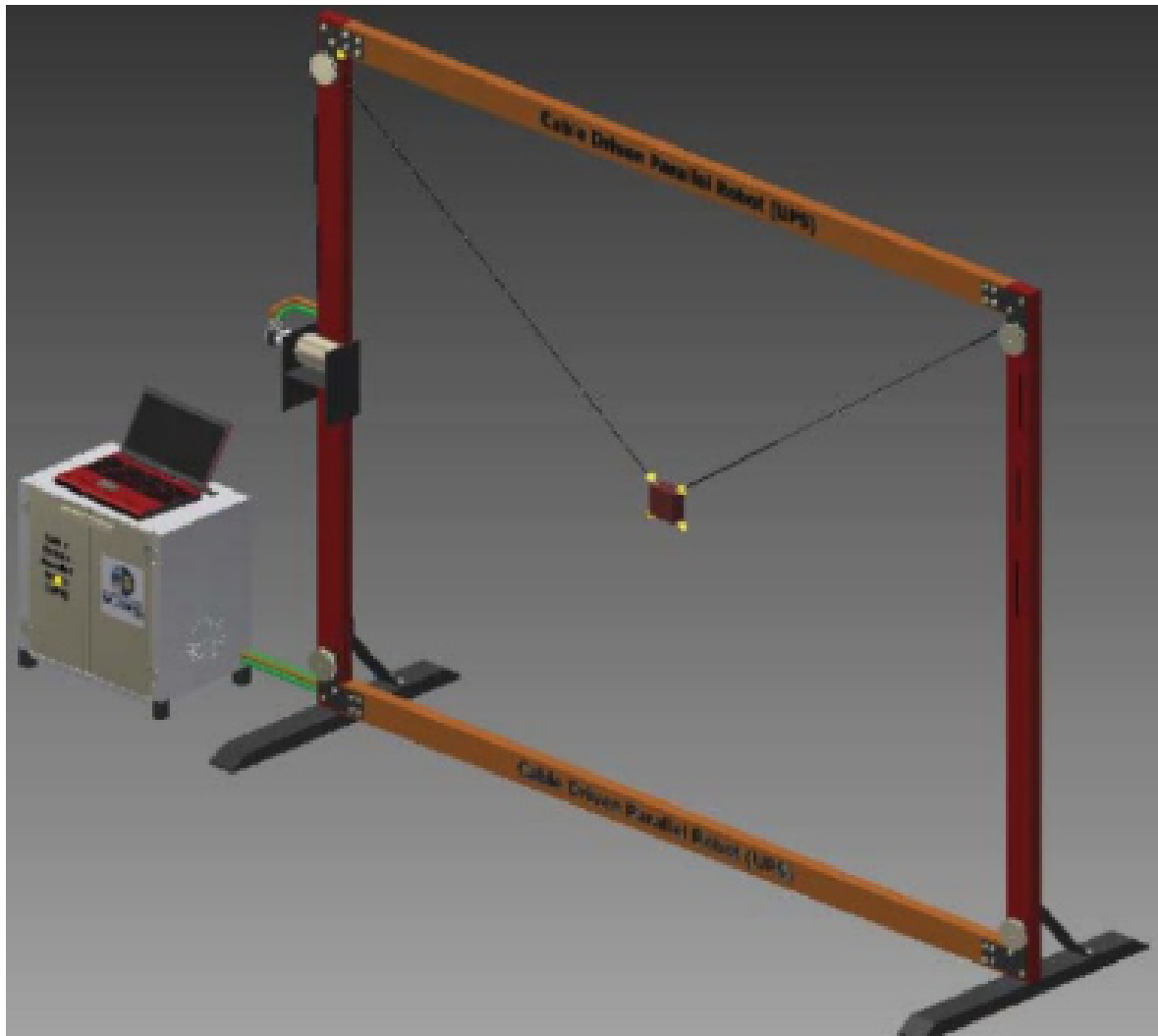


Figura 1.1: Ejemplo de CDPR de dos grados de libertad. En este tipo de sistemas, el movimiento del efector final se obtiene mediante la variación de la longitud de los cables conectados a poleas fijas de la estructura [1].

Y por último, es importante mencionar que la principal motivación que ha llevado a elegir este tipo de robots como tema principal de este trabajo se debe a que el comportamiento global del sistema viene determinado principalmente por la correcta formulación del modelo cinemático, además de todo lo que respecta a la implementación software.

1.2. Planteamiento del problema

El problema principal que se aborda en este trabajo es el control de la posición de un efector final suspendido mediante dos cables ubicados en una estructura fija, en este caso una pizarra móvil con ruedas, a través de poleas, donde el sistema debe ser capaz de desplazar el efector final hacia una o varias posiciones específicas, siendo capaz de definir trayectorias en caso que el efector final pase por más de una posición, todo ello dentro de una región bidimensional cuya área viene definida por la distancia que separa ambas poleas.

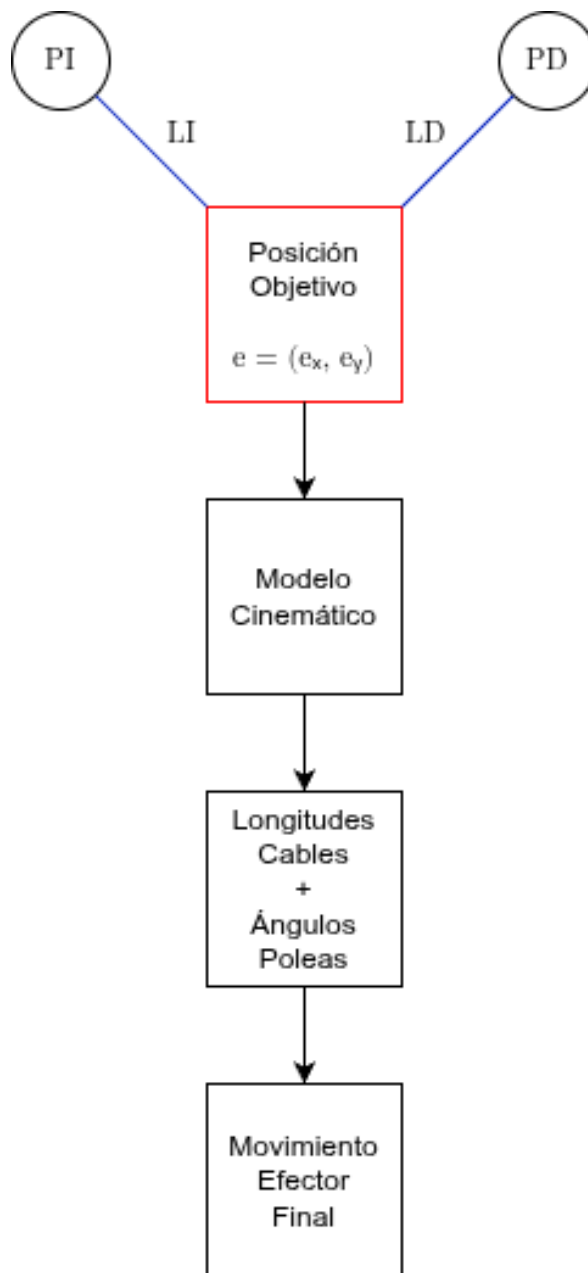


Figura 1.2: Esquema conceptual del problema abordado

Sabiendo esto, es necesario establecer una relación entre la región por la que se mueve el efector final y las variables físicas del sistema, como son las longitudes de los cables y el ángulo que debe rotar cada polea para desplazar el efector final a una posición específica. Esta relación debe ser lo suficientemente robusta de forma que pueda implementarse en un entorno real, y lo suficientemente flexible como para poder integrarla en software robótico.

Este problema en concreto aparece en numerosas aplicaciones. Algunos ejemplos de ello son los sistemas que utilizan las cámaras de retransmisiones deportivas para permanecer suspendidas dentro de los estadios, concretamente las llamadas "spidercam", plataformas para manipular cargas en entornos industriales o sistemas de posicionamiento de herramientas en grandes espacios de trabajo. En todos estos casos se transforma una posición objetivo en órdenes de movimiento para los mecanismos encargados de accionar los cables.

El desarrollo de este trabajo se ha centrado principalmente en desarrollar todo el software necesario para resolver este problema, dejando la implementación del sistema real como una fase a futuro, y que abordaré en conjunto con mi tutor Juan Sebastián Cely Gutiérrez.

1.3. Objetivos

El objetivo principal de este trabajo consiste en el diseño, implementación y validación de un sistema capaz de controlar un CDPR bidimensional, comenzando por la definición del modelo cinemático, pasando por su integración en ROS 2 y finalizando con la preparación de una plataforma experimental orientada a futuras pruebas en un entorno físico real.

Los objetivos específicos de este trabajo son:

- Desarrollo de un modelo cinemático que relacione la posición en la que se encuentre el efector final con las longitudes de los cables y los ángulos de giro de las poleas en cada momento.
- Validación del modelo desarrollado en distintos lenguajes de programación con el fin de comprobar los resultados obtenidos en cada uno de ellos y garantizar así que el comportamiento del modelo es independiente del tipo de lenguaje utilizado.
- Implementación del modelo cinemático dentro de una arquitectura software basada en ROS 2 que permita gestionar movimientos, trayectorias y comunicaciones entre procesos.
- Integración completa del sistema en ROS 2 incorporando mecanismos de visualización y monitorización mediante RViZ.
- Diseño e integración del software desarrollado sobre un sistema hardware real que se tendrá en cuenta como una propuesta de mejora a futuro.

1.4. Alcance y limitaciones

El alcance de este TFG se centra principalmente en la parte software del sistema, dentro de la cual se ha desarrollado y validado un modelo cinemático, además de su integración en ROS 2. Sin embargo, no se ha profundizado en otros aspectos como el modelado de fuerzas, las tensiones de los cables, la estructuración del sistema de forma que este sea de tipo "lazo cerrado", o una interacción más detallada con los sensores en el sistema real, ya que en este caso, éste está compuesto por dos poleas como únicos elementos hardware.

Por tanto, aunque se presente una descripción del sistema hardware, tanto la construcción del robot real como su puesta en funcionamiento quedan algo más apartadas. Todas estas limitaciones han sido asumidas desde el principio, ya que otro de los objetivos principales de este TFG comprende que toda la parte del desarrollo software sea sólida.

1.5. Metodología

1.5.1. Fase 1

La primera fase consiste en analizar el problema y en definir la geometría del sistema, donde se han identificado las variables físicas más relevantes, restricciones geométricas y la relación entre el movimiento del efector final y la longitud de los cables.

1.5.2. Fase 2

Con el sistema ya definido, se ha desarrollado el modelo cinemático inverso para calcular las longitudes de los cables y los ángulos de giro de las poleas a partir de la posición objetivo a la que se quiera desplazar el efector final, lo que ha permitido obtener todas las expresiones matemáticas que componen el núcleo del sistema.

1.5.3. Fase 3

Posteriormente se ha validado este modelo en diferentes implementaciones realizadas en MATLAB, Python y C++, respectivamente. Estas pruebas han permitido comprobar la consistencia de los resultados obtenidos y la detección de posibles errores antes de entrar en fases de desarrollo más avanzadas.

1.5.4. Fase 4

Y por último, el modelo ha sido validado e integrado en una arquitectura software basada en ROS 2, dentro de la cual se ha desarrollado el nodo principal del sistema, la comunicación entre topics y la incorporación de herramientas de visualización en RViZ, lo que ha permitido analizar el comportamiento del sistema.

1.6. Descripción inicial del sistema

Aunque la parte hardware del sistema se tenga en cuenta como una propuesta de mejora a futuro, se ha planteado un diseño consistente en consonancia con el modelo software desarrollado, en el que el sistema está compuesto por una estructura rígida, en este caso una pizarra móvil con ruedas, que soporta dos poleas fijas ubicadas en las esquinas superiores de la misma, a las cuales se conectan los cables encargados de desplazar el efector final.

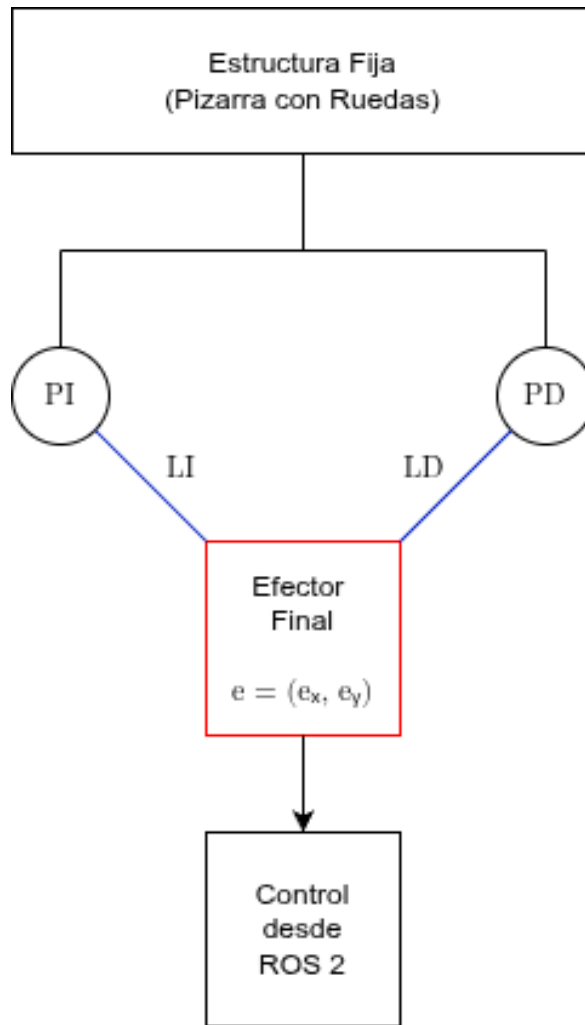


Figura 1.3: Descripción general de los componentes del CDPR

En este diagrama se muestran los elementos principales que componen el sistema y su relación con todos los componentes físicos y software que intervienen en su funcionamiento.

1.7. Herramientas de apoyo empleadas durante el desarrollo

Durante el desarrollo de este trabajo se ha utilizado ChatGPT como única herramienta de Inteligencia Artificial generativa, únicamente como apoyo puntual en tareas de organización, planificación y revisión de contenidos.

El uso de esta herramienta se ha limitado a la consulta de dudas concretas sobre ciertas partes del software implementado, propuestas para una mejor estructuración de la memoria y recomendaciones sobre cómo abordar de la mejor forma posible determinadas tareas.

En ningún caso el tiempo dedicado a la realización de tareas como el desarrollo del modelo cinemático, la implementación del software ni la validación de los resultados obtenidos han sido realizadas de forma automática por esta herramienta.

Toda la información ofrecida y generada por esta herramienta ha sido revisada, contrastada y adaptada antes de ser incorporada al trabajo, manteniendo así en todo momento la responsabilidad sobre el contenido desarrollado y respetando en todo momento los principios académicos de autoría y originalidad.

1.8. Relación con los conocimientos adquiridos en el grado

El desarrollo de este trabajo ha requerido aplicar conocimientos adquiridos en diversas asignaturas del Grado en Ingeniería de Robótica Software. Concretamente, se han empleado conceptos relacionados con programación, arquitectura software, modelado matemático y cinemático, y sobre todo, robótica, que han servido como base para el diseño e implementación del sistema presentado.

Tabla 1.1: Relación entre asignaturas del grado y su aportación al TFG

Asignatura	Conocimientos aplicados en el TFG
Álgebra	Modelado geométrico y representación matemática del sistema CDP.
Cálculo	Cálculo de distancias, relaciones geométricas y formulación del modelo cinemático.
Fundamentos Físicos de la Robótica	Comprensión de magnitudes físicas relacionadas con posición, movimiento y estructura del sistema.
Algoritmos y Estructuras de Datos	Diseño eficiente de funciones y organización de la lógica del sistema.
Diseño Software	Organización modular del proyecto y definición de la arquitectura software.
Sensores y Actuadores	Comprensión del funcionamiento de servomotores y su aplicación en el accionamiento de poleas.
Arquitecturas Software para Robots	Diseño de la arquitectura basada en ROS 2 y comunicación entre componentes.
Modelado y Simulación de Robots	Desarrollo y validación del modelo cinemático mediante simulaciones.
Sistemas Empotrados y de Tiempo Real	Integración futura con hardware físico y consideraciones de ejecución en tiempo real.
Sistemas Distribuidos y Concurrentes	Comunicación mediante topics y organización distribuida de ROS 2.
Planificación y Sistemas Cognitivos	Generación y ejecución de trayectorias a partir de múltiples posiciones objetivo.
Robótica de Servicio	Contextualización del sistema en aplicaciones robóticas reales.
Mecatrónica	Integración entre hardware y software dentro del sistema desarrollado.

La realización de este trabajo ha permitido integrar conocimientos procedentes de diferentes áreas del grado dentro de un único proyecto. Esta unión ha resultado especialmente relevante para abordar tanto el desarrollo del modelo cinemático como la implementación de una arquitectura software basada en ROS 2 y su futura integración en un sistema robótico real.

Capítulo 2

Fundamentos teóricos

2.1. Introducción

Los robots paralelos accionados por cables (CDPR) han experimentado un gran crecimiento en los últimos años. Comprenden conceptos de robótica paralela, modelado geométrico y mecánica ligera, convirtiéndolos en un tipo de robots muy atractivo tanto para proyectos de investigación como para aplicaciones prácticas.

Este capítulo presenta el contexto teórico que comprende este TFG, además de abordar conceptos fundamentales sobre los CDPR, sus configuraciones más habituales y sus principales líneas de investigación, todo ello relacionado con el enfoque software adaptado.

2.2. Robots paralelos accionados por cables (CDPR)

Un CDPR se caracteriza por utilizar cables como elementos para transmitir el movimiento entre la estructura y el efector final. Esta característica introduce una serie de particularidades desde el punto de vista del modelado y del control del sistema, ya que es necesario garantizar que los cables permanezcan tensionados en todo momento. A pesar de ello, este tipo de robots tiene un peso estructural mucho menor, una mayor escalabilidad y la posibilidad de desplazarse por regiones bastante amplias.

2.3. Configuraciones típicas

Los CDPR pueden adoptar múltiples configuraciones en función del número de cables, puntos de anclaje y grados de libertad por los que se mueve el efector final. En caso de que se quiera mover el efector final en tres dimensiones, el número de cables debe ser superior al número de grados de libertad, con el fin de garantizar que todos los cables utilizados se encuentren tensionados.

En este caso, el efector final se mueve en dos dimensiones, por lo que el número de cables debe ser como mínimo igual al número de grados de libertad, condición que se cumple satisfactoriamente desde el principio.

2.4. Modelo cinemático

El modelo cinemático de un CDPR tiene como objetivo establecer una relación entre la posición del efector final y las longitudes de los cables que lo accionan en cada momento, todo ello haciendo uso de las distancias euclídeas entre cada polea y el efector final.

En este caso, se ha optado por trabajar directamente con la cinemática inversa, ya que lo que se quiere es calcular las longitudes de cada cable siendo conocida la posición del efector final en cada momento. Este enfoque resulta adecuado para su posterior implementación software e integración en ROS 2.

2.5. Control y planificación de trayectorias

El control de un CDPR presenta diversos retos en lo que respecta a los cables y a la continua necesidad de mantenerlos en tensión, ya que en sistemas más avanzados, lo más habitual es incorporar un modelo dinámico, un sistema de control en lazo cerrado y otras estrategias centradas en la tensión de los cables.

Sin embargo, al tratarse de una aplicación mucho más sencilla, en la que el objetivo principal consiste en la validación del comportamiento geométrico del sistema, lo normal es utilizar un modelo cinemático y un sistema de control en lazo abierto, donde la planificación de trayectorias se realice a partir de un conjunto de posiciones objetivo y los cálculos necesarios para pasar por todas ellas de forma continua.

2.6. Frameworks robóticos

ROS 2 se considera actualmente como uno de los frameworks de referencia en el desarrollo de sistemas robóticos, ya que su arquitectura basada en nodos, topics y servicios, hacen mucho más fácil y llevadera la creación de este tipo de sistemas.

Es por eso que la integración de un CDPR en ROS 2 permite separar y distinguir sin ningún problema el modelo cinemático, la lógica de control y la visualización del sistema simulado, la cual se llevará a cabo en RViZ.

2.7. Aplicaciones y líneas de investigación actuales

Los CDPR son utilizados en la actualidad desde sistemas de posicionamiento, manipulación de grandes cargas y plataformas de movimiento, hasta algunos robots de rehabilitación.

Por otro lado, las principales líneas de investigación actuales están centradas en el análisis del espacio de trabajo, la optimización de las tensiones de los cables que componen el sistema, lograr una mayor robustez en el control del sistema y garantizar una interacción segura en todo su entorno.

2.8. Relación con el trabajo de fin de grado desarrollado

Este TFG se sitúa dentro de todo este contexto desde un enfoque simplificado en comparación con aplicaciones ya existentes, haciendo énfasis en el desarrollo software, el modelo cinemático y la integración en ROS 2, lo que permite sentar unas bases sólidas de forma que, en un futuro, se pueda añadir un modelo dinámico, el uso de otro tipo de sensores aparte de los servomotores que actúan como poleas, o la implementación de un sistema de control mucho más avanzado.

Capítulo 3

Plataforma de desarrollo

3.1. Introducción

En este capítulo se presenta una visión global del sistema desarrollado en este TFG, ofreciendo una descripción del CDPR diseñado y de toda la arquitectura que hace posible su funcionamiento, la cual ha permitido validar el modelo cinemático, así como los algoritmos para planificar trayectorias utilizados y su integración en ROS 2. Y aunque gran parte del sistema hardware se encuentre en una fase preliminar, todo su diseño se está teniendo en cuenta desde el principio.

3.2. Funcionamiento de un CDPR bidimensional

El funcionamiento principal de un CDPR bidimensional consiste en el desplazamiento de un efector final, dentro de una región del mismo tipo que el robot, todo ello mediante el accionamiento de dos cables, donde cada uno de ellos conecta el efector final con dos poleas fijas, situadas en cada una de las esquinas superiores de la estructura, lo que permite controlar su posición en función de la longitud de cada uno de los cables.

Desde un punto de vista funcional, el sistema recibe como entrada una o varias posiciones objetivo, a partir de las cuales el sistema va calculando las longitudes de los cables y los ángulos de rotación de cada polea necesarios para hacer que el efector final pase por todas y cada una de las posiciones recibidas.

3.3. Arquitectura

La arquitectura del sistema sigue un enfoque modular y escalable. Esto significa que, aunque toda su implementación abarque un único nodo, posee un diseño que separa las diferentes capas que componen el sistema, las cuales se describen a continuación:

- *Capa de entrada:* Define la posición o conjunto de posiciones (trayectorias) objetivo por las que el efector final debe pasar.
- *Capa cinemática:* Aplica el modelo cinemático para transformar las posiciones recibidas en parámetros físicos del sistema.
- *Capa visual:* Muestra el estado del sistema en cada momento, lo que permite analizar su comportamiento y comprobar que todo está funcionando correctamente.

Toda esta arquitectura facilita la comprensión del sistema y hace que, en caso de que se quiera realizar una ampliación del mismo en un futuro, ésta se pueda llevar a cabo de una forma mucho más sencilla.

Con el objetivo de visualizar el sistema descrito anteriormente, se adjunta a continuación un diagrama que representa la arquitectura general del sistema y la relación entre sus componentes principales.

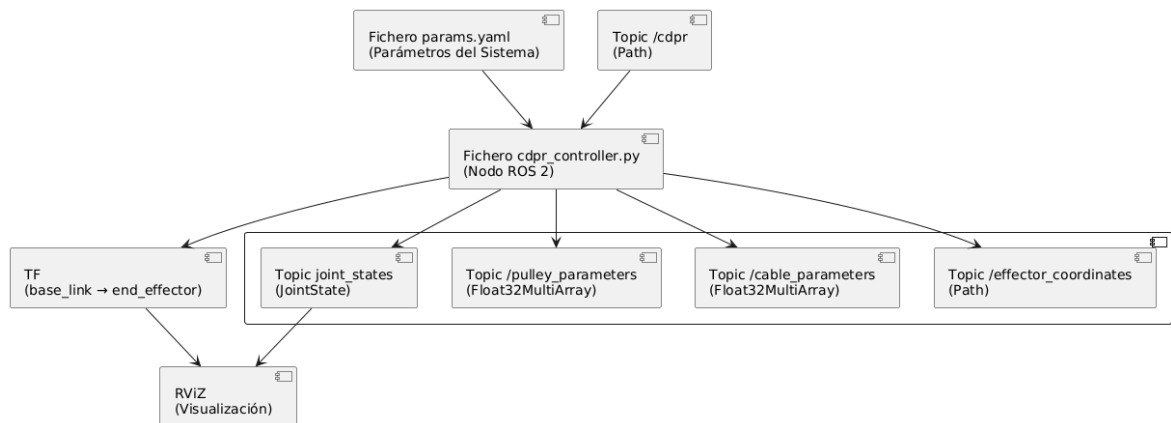


Figura 3.1: Arquitectura general del sistema en ROS 2

Como se puede observar, se tiene una única posición o un conjunto de posiciones objetivo definidas por el usuario, las cuales se envían al nodo principal del sistema implementado, encargado de aplicar el modelo cinemático del robot para calcular las longitudes y los ángulos respecto a la vertical de cada cable, al igual que los ángulos de rotación y la cantidad de cable elongada o recogida por cada polea.

Una vez calculados estos parámetros, toda esta información se publica mediante topics, lo que permite a otros componentes del sistema poder utilizar esta información, mientras se visualiza en RViZ el estado del sistema en cada momento utilizando el modelo URDF definido, lo que permite observar el comportamiento de todos los elementos que componen el sistema durante la ejecución de una trayectoria.

3.4. Componentes software

El desarrollo software es lo que compone en su gran mayoría al núcleo de este TFG, el cual se ha llevado a cabo en Python, ya que es un lenguaje de programación de alto nivel que se integra perfectamente con ROS 2 y que se caracteriza principalmente por su flexibilidad y legibilidad en código, siendo éstos los factores principales que he tenido en cuenta a la hora de elegir este lenguaje de programación antes que otros, como han sido MATLAB o C++.

El sistema software está compuesto principalmente por los siguientes elementos, los cuales trabajan de forma coordinada con el objetivo de ofrecer una simulación bastante coherente:

- Un nodo ROS 2 encargado de gestionar la ejecución del sistema, el cual recibe como entrada la posición o la trayectoria por la que se quiere desplazar el efector final.
- Un fichero de configuración que permite ajustar los parámetros del sistema y personalizar la visualización.
- Un modelo descriptivo del robot en URDF utilizado para su representación en RViZ.

3.5. Sistema hardware

El sistema hardware comprende una estructura rígida, en este caso una pizarra móvil con ruedas, encargada de sujetar los dos servomotores situados en las esquinas superiores de la misma que actúan como poleas, que a su vez se encargan de accionar los cables que definen la posición del efector final en cada momento, el cual se desplaza libremente dentro de la región bidimensional definida por el área de la pizarra.

Aunque la construcción física del sistema real no esté completa a día de hoy, todo su diseño se ha definido en consonancia con todo el software desarrollado, ya que la posición de cada una de las poleas, el espacio de trabajo equivalente a las dimensiones de la pizarra y el tamaño del efector final se han tenido en cuenta desde las primeras fases de este TFG.

3.6. Flujo general de funcionamiento

En primer lugar, se recibe como entrada del sistema una o varias posiciones objetivo por las que el efector final debe pasar, a partir de las cuales el sistema aplica el modelo cinemático, obteniendo así todos los parámetros necesarios que permiten al sistema accionar los cables con el objetivo de que el efector final pase por todas las posiciones recibidas.

Con todos estos parámetros ya calculados, el sistema se va actualizando en cada momento mientras va publicando información relevante, como la posición del efector final, el valor de la longitud de cada cable y el ángulo girado por cada polea en tiempo real. Todo este flujo de trabajo se repite continuamente hasta que el efector final llegue a la última posición que haya recibido por su entrada.

Es por ello que, para representar el comportamiento del sistema durante su ejecución, se adjunta a continuación un diagrama de flujo que describe todo este proceso.

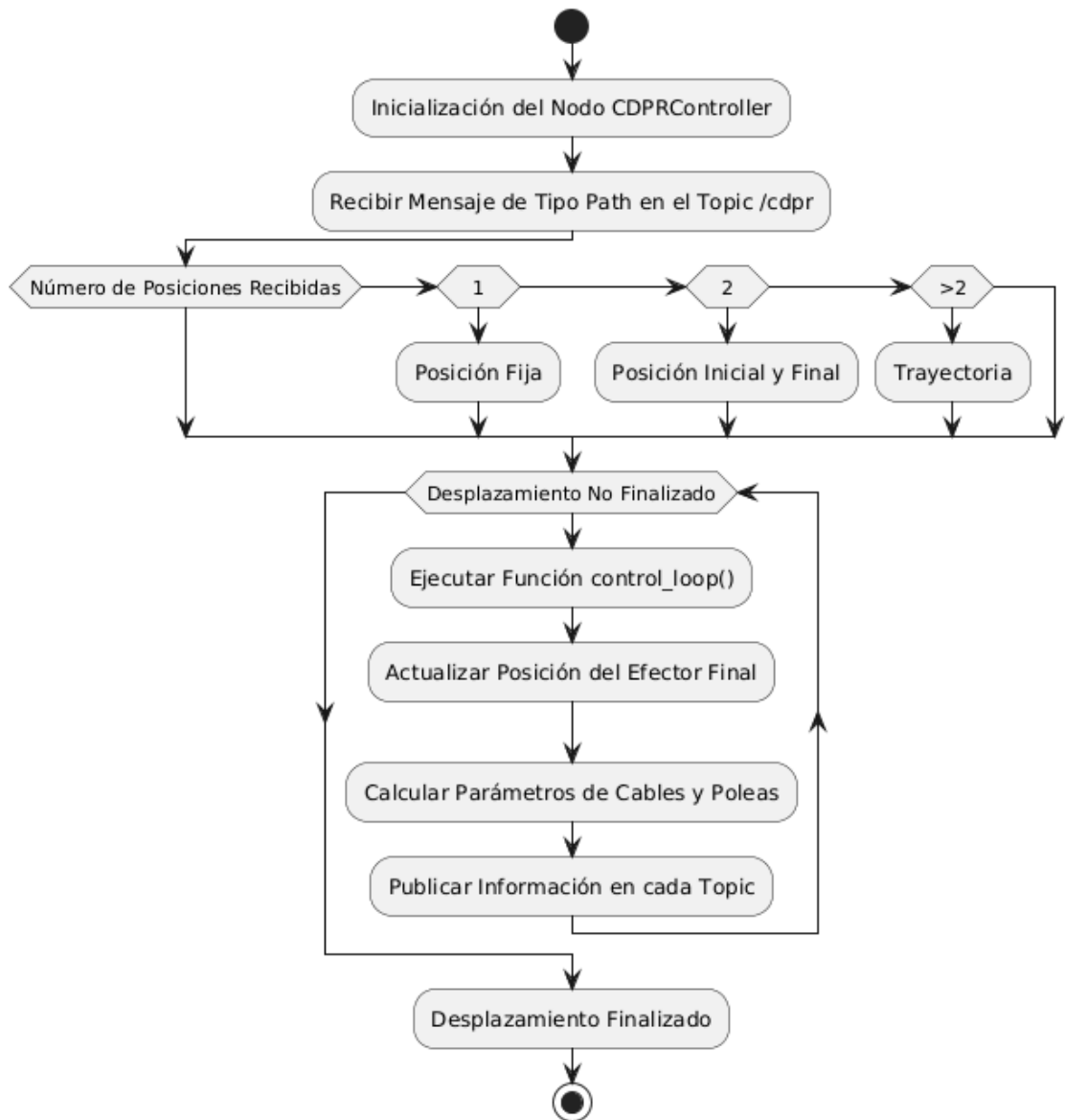


Figura 3.2: Flujo general de funcionamiento del sistema

Como se puede observar, el nodo recibe una única posición o una trayectoria objetivo, a partir de la cual se aplica el modelo cinemático para calcular la longitud de cada cable y los ángulos de rotación de cada polea.

Una vez hecho esto, el sistema publica los resultados obtenidos en topics, permitiendo así conocer el estado del sistema en tiempo real, al igual que en RViZ, donde se puede observar el movimiento del efector final y de las poleas en cada momento de la ejecución del sistema.

3.7. Relación entre módulos

Aunque gran parte de toda la funcionalidad del sistema se implemente en un único nodo, se pueden identificar claramente las relaciones entre todos los módulos que lo componen, como son las posiciones recibidas por su entrada, el cálculo cinemático y su visualización, lo que permite analizar y depurar el comportamiento del sistema de una forma mucho más estructurada, y que además, esta organización se puede seguir teniendo en cuenta en caso de que se quieran hacer ampliaciones en un futuro, como podría ser la adición de otro tipo de sensores en adición a los servomotores que ya se tienen, el paso de tener un sistema de control en lazo abierto a cerrado, o la separación del código en múltiples nodos ROS 2 en lugar de uno solo.

Capítulo 4

Modelo cinemático

El modelo cinemático constituye uno de los elementos fundamentales de este TFG, ya que permite establecer una relación entre la posición del efector final y las variables físicas que controlan su movimiento.

En un CDPR, el posicionamiento del efector final se consigue a partir de la variación de la longitud de cada uno de los cables conectados a las poleas situadas en las esquinas superiores de la estructura fija del sistema. Esto hace necesario definir un modelo geométrico que permita calcular, para cualquier posición en la que se encuentre el efector final, las longitudes de los cables y los ángulos que formen cada uno de ellos respecto a la vertical.

El modelo desarrollado en este TFG corresponde a un CDPR bisimensional de dos grados de libertad, formado por dos cables, dos poleas situadas en las esquinas superiores de una estructura fija, y un efector final rígido que se desplaza dentro de un espacio de trabajo bidimensional. Es por ello que, en este caso, se ha optado por un modelo puramente geométrico basado en cinemática inversa, donde no se han considerado efectos dinámicos como la masa del efector final, la elasticidad de los cables o la fricción de las poleas.

4.1. Descripción geométrica del sistema

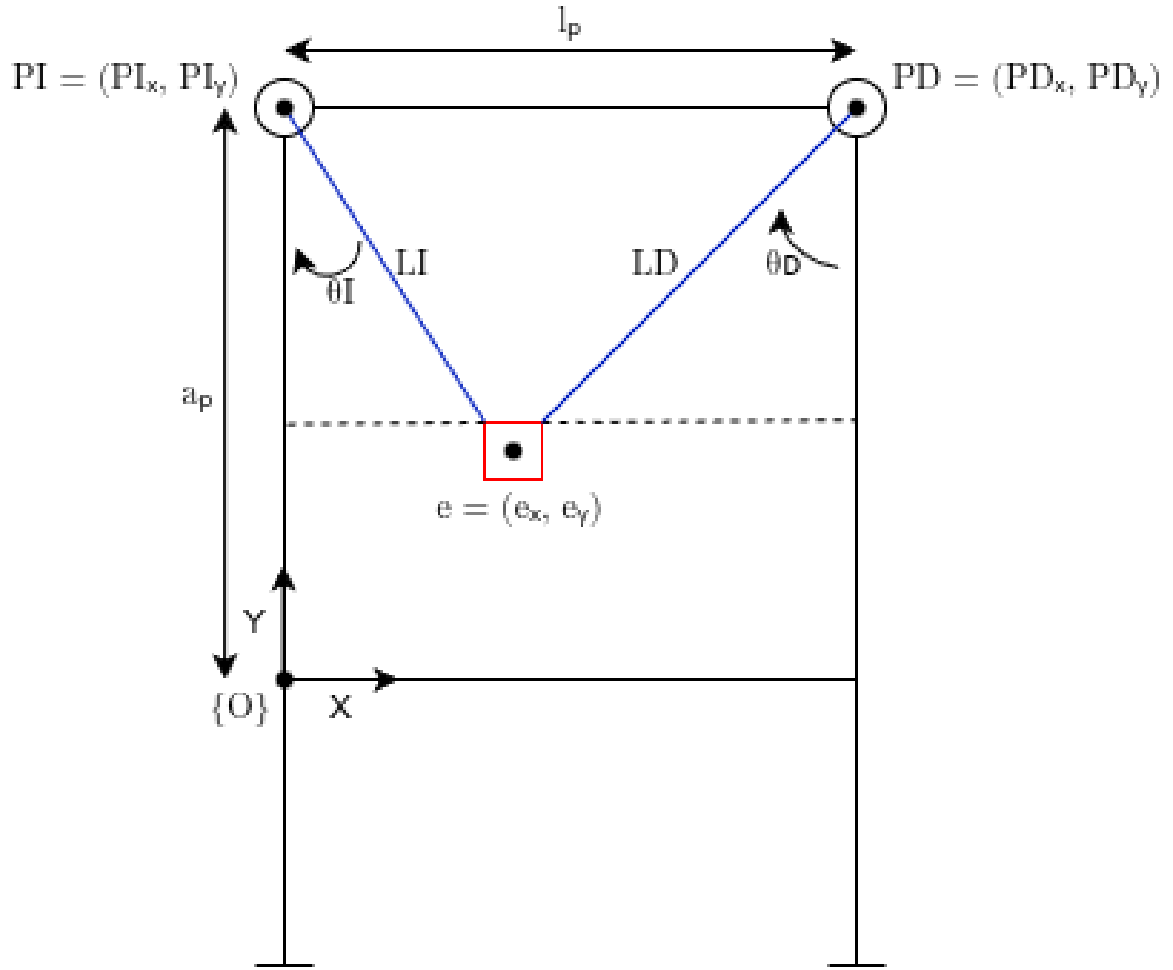


Figura 4.1: Modelo cinemático de un CDPR bidimensional

En esta imagen se representa un sistema de referencia cartesiano fijo $\{O\}$ situado en la esquina inferior izquierda de la estructura.

Las dimensiones del espacio de trabajo vienen definidas por:

- l_p : Largo del espacio de trabajo.
- a_p : Alto del espacio de trabajo.

Las poleas se encuentran situadas en las esquinas superiores de la estructura y sus posiciones son conocidas y constantes durante todo el funcionamiento del sistema.

La posición del efector final viene definida de la siguiente forma:

$$e = (e_x, e_y) \quad (4.1)$$

Ecuación 4.1: Posición del efector final en el espacio de trabajo

donde:

- e_x representa la coordenada horizontal del efector final.
- e_y representa la coordenada vertical del efector final.

De la misma forma, el efector final también posee un par de dimensiones físicas que deben tenerse en cuenta al formular el modelo:

- l_e : Largo del efector final.
- a_e : Alto del efector final.

Todas estas dimensiones permiten modelar correctamente los puntos reales de unión entre los cables y el efector final, mejorando así la precisión geométrica del sistema en comparación con otros modelos más simplificados que pueden considerar al efector final como un simple punto material.

4.2. Definición de poleas y parámetros conocidos

Las coordenadas en las que se encuentran cada una de las poleas son parámetros fijos del sistema.

La polea izquierda se define como:

$$PI = (PI_x, PI_y) = (0, a_p) \quad (4.2)$$

Ecuación 4.2: Posición de la polea izquierda en el espacio de trabajo

Mientras que la polea derecha queda definida como:

$$PD = (PD_x, PD_y) = (l_p, a_p) \quad (4.3)$$

Ecuación 4.3: Posición de la polea derecha en el espacio de trabajo

Estas posiciones corresponden a la configuración mecánica prevista para un prototipo físico, donde ambas poleas quedarían situadas en cada una de las esquinas superiores de la estructura fija.

4.3. Geometría del efector final

En vez de plantearlo como una aproximación puntual, el efector final se modela como un cuerpo rígido rectangular cuyas dimensiones son l_e y a_e .

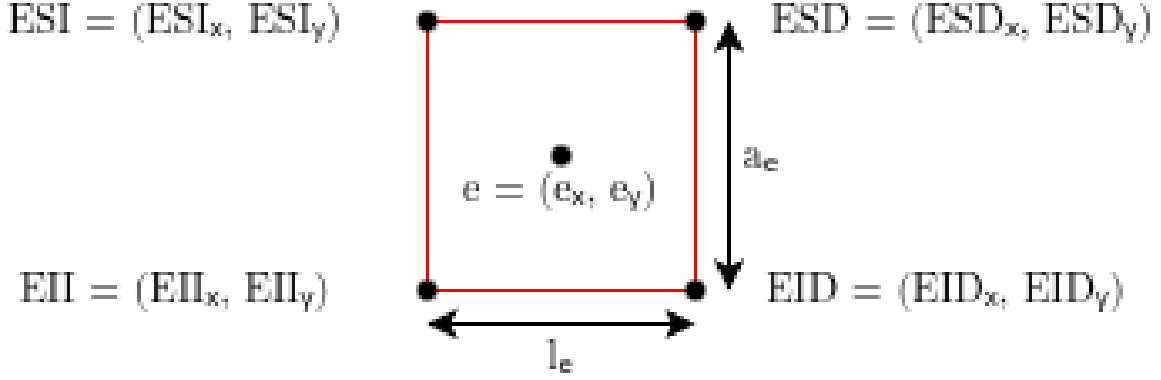


Figura 4.2: Geometría y dimensiones del efector final

Sabiendo que la posición del centro del efector final en el espacio de trabajo es $e = (e_x, e_y)$, se pueden determinar las posiciones de cada una de sus cuatro esquinas.

$$ESI = (ESI_x, ESI_y) = (e_x - \frac{l_e}{2}, e_y + \frac{a_e}{2}) \quad (4.4)$$

Ecuación 4.4: Posición de la esquina superior izquierda del efector final

$$ESD = (ESD_x, ESD_y) = (e_x + \frac{l_e}{2}, e_y + \frac{a_e}{2}) \quad (4.5)$$

Ecuación 4.5: Posición de la esquina superior derecha del efector final

$$EII = (EII_x, EII_y) = (e_x - \frac{l_e}{2}, e_y - \frac{a_e}{2}) \quad (4.6)$$

Ecuación 4.6: Posición de la esquina inferior izquierda del efector final

$$EID = (EID_x, EID_y) = (e_x + \frac{l_e}{2}, e_y - \frac{a_e}{2}) \quad (4.7)$$

Ecuación 4.7: Posición de la esquina inferior derecha del efector final

Dado que los cables están unidos a las esquinas superiores del efector final, ESI y ESD son las únicas que intervienen directamente en cálculos cinemáticos posteriores.

4.4. Cinemática inversa

El problema de cinemática inversa que se tiene en este sistema consiste en determinar las longitudes que debe tener cada uno de los cables para posicionar el efector final en las coordenadas deseadas dentro del espacio de trabajo.

Para ello, se calcula la distancia euclídea entre cada polea y la esquina superior correspondiente del efector final.

$$LI = \sqrt{(ESI_x - PI_x)^2 + (ESI_y - PI_y)^2} \quad (4.8)$$

Ecuación 4.8: Longitud del cable izquierdo

$$LD = \sqrt{(ESD_x - PD_x)^2 + (ESD_y - PD_y)^2} \quad (4.9)$$

Ecuación 4.9: Longitud del cable derecho

Estas ecuaciones permiten obtener directamente la longitud que debe tener cada cable para llegar a cualquier posición que sea alcanzable por el efector final.

Desde el punto de vista software, estas expresiones constituyen el núcleo del modelo cinemático implementado en pruebas posteriormente realizadas en MATLAB, Python, C++ y ROS 2.

4.5. Cálculo de la orientación de los cables

Además de la longitud de cada cable, es interesante conocer la orientación que presenta cada uno respecto a la vertical, ya que esta información resulta muy útil para:

- Analizar la configuración geométrica del sistema.
- Representar correctamente los cables en RViZ (no de forma visual pero sí de forma matemática).
- Calcular posteriormente otros parámetros relacionados con el accionamiento de las poleas, como por ejemplo la cantidad de cable recogido cuando el efector se ha desplazado entre varias posiciones.

El ángulo del cable izquierdo se define como:

$$\theta_I = -\arctan\left(\frac{ESI_x - PI_x}{ESI_y - PI_y}\right) \quad (4.10)$$

Ecuación 4.10: Ángulo que forma el cable izquierdo con la vertical

Mientras que el ángulo del cable derecho viene definido por:

$$\theta_D = \arctan\left(\frac{ESD_x - PD_x}{ESD_y - PD_y}\right) \quad (4.11)$$

Ecuación 4.11: Ángulo que forma el cable derecho con la vertical

Los ángulos obtenidos se calculan inicialmente en grados, que posteriormente pueden convertirse a radianes para hacer más sencilla tanto la interpretación de los resultados obtenidos como la visualización gráfica del sistema.

4.6. Relación entre cables y poleas

El accionamiento del sistema se lleva a cabo mediante el uso de dos poleas encargadas de recoger o liberar una longitud de cable determinada, en función de la posición a la que se quiera desplazar el efector final.

Si R_p representa el radio de la polea y ΔL representa la variación de longitud experimentada por un cable entre dos posiciones consecutivas, el desplazamiento angular correspondiente se puede obtener mediante la siguiente ecuación:

$$\Delta\theta = \frac{\Delta L}{R_p} \quad (4.12)$$

Ecuación 4.12: Relación longitud cable - radio polea

donde:

- $\Delta\theta$ es el giro angular de la polea.
- ΔL es la variación de longitud del cable.
- R_p es el radio de la polea.

Esta ecuación permite establecer la transformación entre el espacio cartesiano del efector final y el espacio articular del sistema, el cual viene definido por los giros realizados por cada una de las poleas.

4.7. Espacio de trabajo y restricciones geométricas

Aunque el modelo matemático permite calcular soluciones para cualquier posición (e_x, e_y) del efector final, existe una serie de restricciones geométricas que deben cumplirse para garantizar que el funcionamiento del sistema sea correcto:

- El efector final debe estar dentro de los límites físicos de la estructura.
- Los cables deben permanecer tensionados.
- Nunca puede haber longitudes de cable negativas.
- El efector final no debe colisionar ni con las poleas ni con la estructura.
- Las poleas deben operar dentro de sus límites mecánicos de giro.

Estas restricciones delimitan el espacio de trabajo del sistema y deben tenerse en cuenta siempre que se quiera planificar trayectorias.

4.8. Consideraciones y limitaciones del modelo

El modelo cinemático presentado constituye una representación simplificada del comportamiento del sistema, adecuado a los objetivos establecidos en este TFG.

Es por ello que se han tenido en cuenta las siguientes limitaciones:

- Los cables son inextensibles y sin masa.
- Las poleas no poseen fricción ni efectos dinámicos y se modelan con un radio constante.
- La orientación del efector final siempre es constante.

A pesar de estas limitaciones, el modelo reproduce correctamente la relación geométrica existente entre la posición del efector final y las variables físicas del sistema, proporcionando una base sólida para la simulación y la integración en ROS 2.

Capítulo 5

Simulación y validación del modelo

5.1. Introducción

Una vez desarrollado el modelo cinemático del CDPR, se ha llevado a cabo un proceso de validación y simulación con el objetivo de comprobar las expresiones matemáticas obtenidas del modelo cinemático en una primera implementación software. Esto permite verificar el comportamiento del sistema para así poder detectar posibles errores de planteamiento y evitar que se vayan acarreando a la hora de integrar el sistema completo en ROS 2.

En el caso de este TFG, las pruebas del modelo cinemático se han ido realizando de forma progresiva mediante su implementación en tres lenguajes de programación diferentes: MATLAB, Python y C++.

5.2. Simulación inicial en MATLAB

MATLAB es el primer lenguaje de programación que se ha utilizado para validar el modelo cinemático, ya que éste se considera idóneo para tratar expresiones matemáticas y obtener una rápida visualización de los resultados. Es por ello que, en esta fase inicial, la simulación se ha centrado exclusivamente en verificar que el efector final se ubica correctamente en una posición (x, y) específica.

Esta primera implementación permite introducir directamente la posición (x, y) en la que se quiere mostrar el efector final, además de ofrecer una representación gráfica de su posición exacta dentro del espacio de trabajo junto con toda la estructura del sistema y la disposición de los cables.

Robot por cables para el control de un efector final

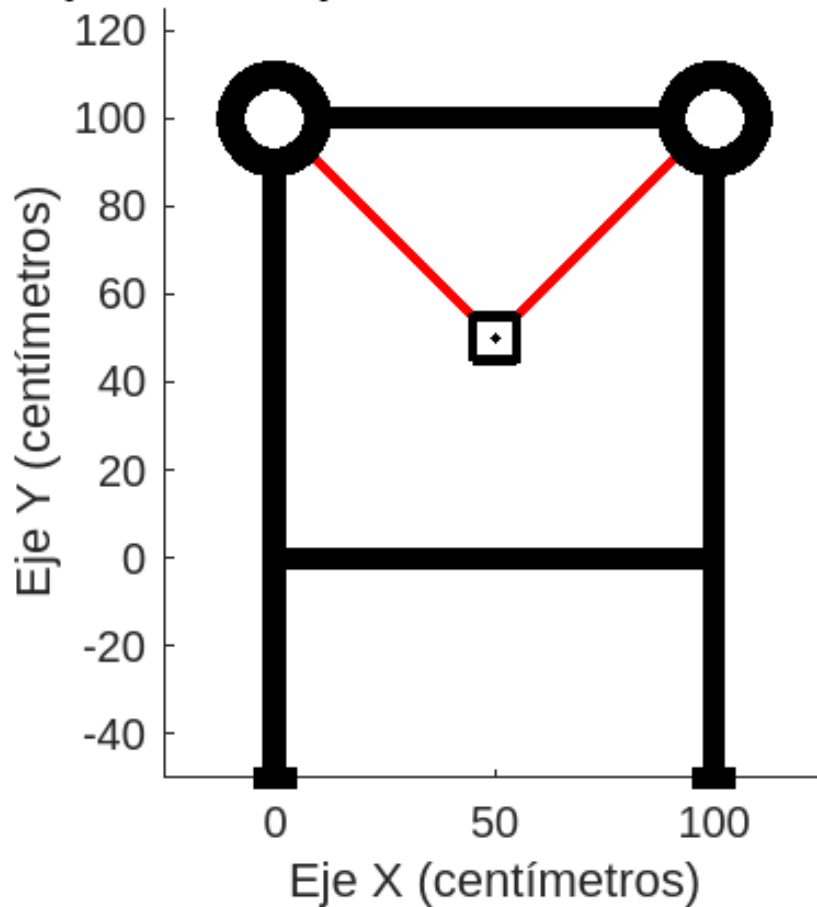


Figura 5.1: Simulación inicial en MATLAB

Con esto se ha cumplido el objetivo de esta primera versión, que no comprende la simulación del movimiento del efector final ni la planificación de trayectorias, sino que lo que se busca es verificar que el modelo cinemático sitúe al efector final en la posición deseada y que las longitudes de los cables calculadas sean coherentes con la geometría del sistema, sirviendo así como validación conceptual del modelo y como punto de partida para implementaciones posteriores.

5.3. Implementación del modelo en Python

Una vez validado el primer modelo en MATLAB, se ha desarrollado una implementación más avanzada en Python, la cual ya incluye animaciones y la posibilidad de definir múltiples posiciones objetivo para así generar trayectorias continuas que conecten dichos puntos.

Esta segunda implementación permite introducir una secuencia de posiciones objetivo para el efector final dentro del espacio de trabajo. A partir de estas posiciones, se pueden interpolar trayectorias rectilíneas entre puntos consecutivos, mientras se calculan en cada instante las longitudes y los ángulos de cada cable utilizando las ecuaciones correspondientes de cinemática inversa.

Además, esta implementación incluye una representación gráfica del sistema, en la que se muestra la estructura, la trayectoria realizada, y la variación del movimiento del efector final y los cables en todo momento, facilitando así la comprensión del funcionamiento del sistema y permitiendo observar de forma intuitiva cómo va evolucionando el movimiento del efector final y de los cables. Es por ello que esta segunda versión ya se aproxima mucho más al funcionamiento que se implementará posteriormente en ROS 2.

Para una mejor comprensión del comportamiento dinámico del sistema mostrado en la figura, se adjunta un enlace de descarga a un vídeo demostrativo al que se puede acceder haciendo clic [aquí](#).

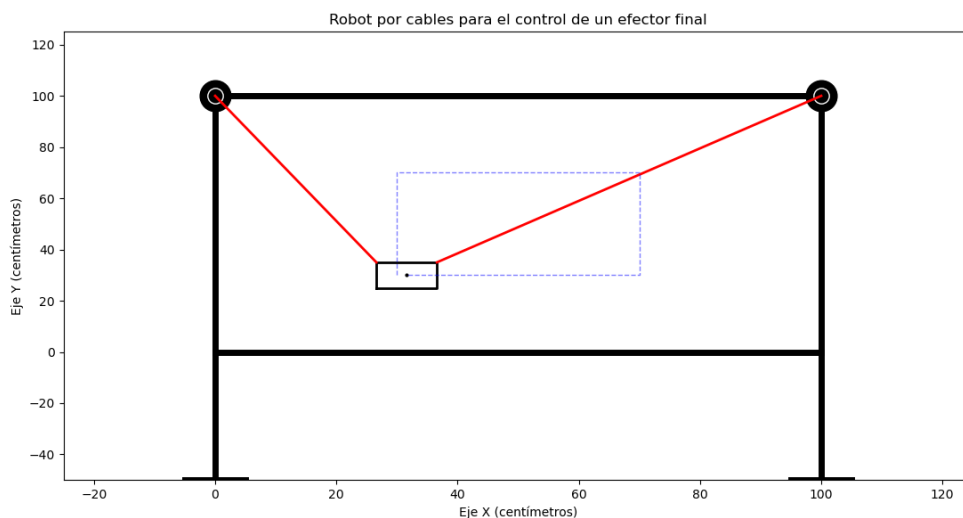


Figura 5.2: Implementación y simulación en Python

5.4. Implementación del modelo en C++

En paralelo a la implementación en Python, se ha desarrollado una versión del modelo cinemático en C++. Esta implementación reproduce la misma lógica que la vista en Python, permitiendo la definición de varias posiciones objetivo y el cálculo de la longitud de los cables en cada momento para recorrer una trayectoria.

Esta simulación se ha llevado a cabo únicamente con fines de verificación y comparación para asegurar que el modelo cinemático es independiente del lenguaje de programación utilizado, y que todas las ecuaciones que lo forman sean consistentes. Y aunque la visualización gráfica en C++ no sea tan completa como la de Python, también es útil para validar todos los cálculos numéricos realizados.

Para una mejor comprensión del comportamiento dinámico del sistema mostrado en la figura, se adjunta un enlace de descarga a un vídeo demostrativo al que se puede acceder haciendo clic [aquí](#).

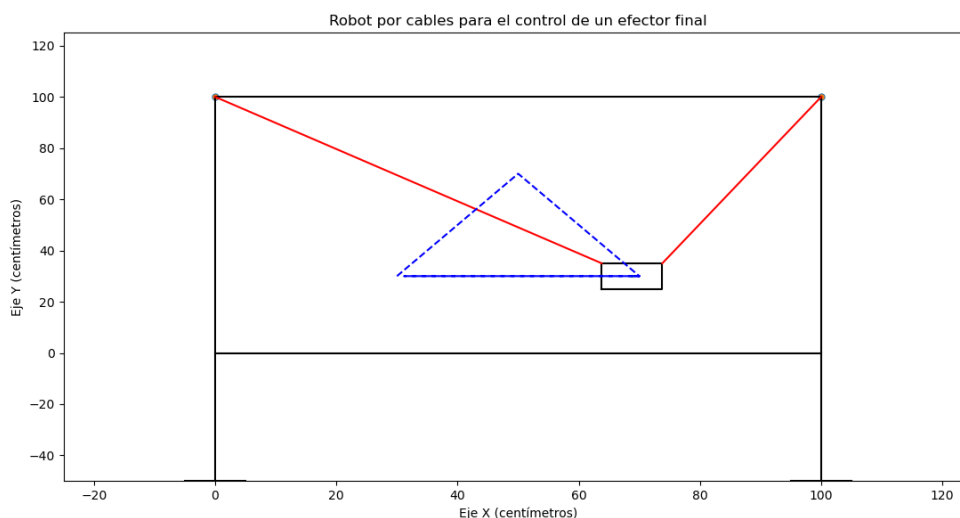


Figura 5.3: Implementación y simulación en C++

5.5. Comparación y validación de resultados

Para validar el modelo cinemático, se han comparado los resultados obtenidos en las tres implementaciones, y todas ellas presentan una muy buena concordancia, lo que deja más que confirmado que el modelo cinemático utilizado es totalmente correcto.

Sin embargo, existen algunas diferencias en cuanto a funcionalidad y flexibilidad. MATLAB se ha utilizado como herramienta para llevar a cabo una primera validación, limitándose únicamente al posicionamiento de una única posición objetivo, mientras que las implementaciones realizadas en Python y en C++ permiten la generación de trayectorias definidas por múltiples posiciones objetivo.

Y es por eso que, tras este análisis comparativo, he decidido utilizar Python como lenguaje principal a la hora de implementar el sistema en ROS 2, debido a la facilidad que supone desarrollar en Python ofreciendo la posibilidad de poder integrarlo directamente en ROS 2, manteniendo el mismo comportamiento cinemático que en C++.

5.6. Análisis de errores y limitaciones

Durante el proceso de simulación, se han identificado algunas limitaciones que no afectan al sistema debido al enfoque que se le ha dado al mismo, ya que al tratarse de un modelo puramente cinemático, no se consideran efectos dinámicos como la masa del efector final, la elasticidad de los cables o la fricción de las poleas.

Por otro lado, todas las simulaciones se realizan en lazo abierto, lo que implica que todos los resultados obtenidos dependen directamente de la precisión del modelo y de los parámetros geométricos definidos. Todas estas limitaciones no invalidan ningún aspecto del sistema ya definido, pero es importante tenerlas en cuenta a la hora de interpretar los resultados cuando se esté implementando en un entorno físico real.

Capítulo 6

Integración del sistema en ROS 2

6.1. Introducción

Una vez validado el modelo cinemático del CDPR en todas las simulaciones mencionadas en el capítulo anterior, el siguiente paso consiste en la integración completa del sistema en ROS 2 (Robot Operating System 2), lo que va a permitir dotar al sistema de una arquitectura modular y escalable, facilitando así su visualización a la hora de probarlo en un sistema físico real.

En este capítulo se describe la estructura general del paquete ROS 2 desarrollado, el funcionamiento del nodo que implementa toda la parte de control del CDPR, todo lo relacionado con las comunicaciones realizadas mediante topics y la visualización del sistema en RViZ.

Toda esta integración se ha llevado a cabo haciendo uso de la distribución Jazzy de ROS 2 y de Python como lenguaje de programación, tal y como se mencionó en el capítulo anterior.

6.2. Estructura del paquete ROS 2

El sistema se ha organizado en un paquete ROS 2 llamado *cdpr_2d*, cuya organización y estructura siguen un modelo estándar, facilitando así la comprensión del código y lo que permite en un futuro modificar o ampliar componentes concretos del sistema sin tener que realizar grandes cambios.

En este paquete ROS 2 se pueden distinguir como elementos principales:

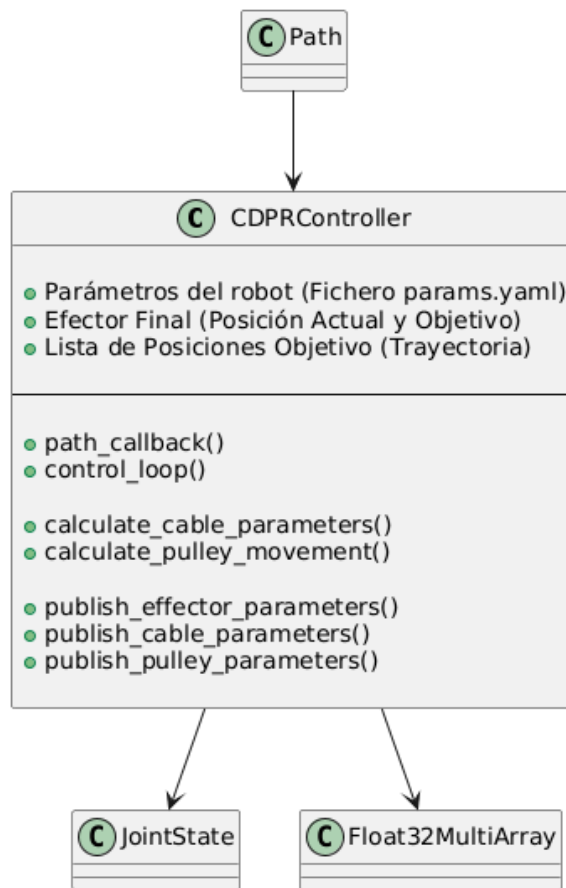
- Fichero *cdpr_controller.py*: Nodo principal que implementa el funcionamiento completo del sistema que incluye la planificación de trayectorias, todos los cálculos cinemáticos y la publicación de resultados haciendo uso de diferentes topics.
- Directorio *config/*: Contiene un fichero de configuración llamado *params.yaml* en el que se definen los parámetros del sistema, y otro fichero llamado *robot.rviz* encargado de configurar la visualización del sistema en RViZ.
- Directorio *description/*: Contiene los modelos del sistema en URDF en el fichero *robot.urdf*, y en XACRO en el fichero *robot.xacro*.
- Directorio *launch/*: Contiene un fichero de lanzamiento llamado *robot_state_publisher.launch.py*, que permite inicializar de forma conjunta tanto la simulación como la visualización del sistema en RViZ.
- Directorio *test/*: Contiene varios ficheros de test automáticos para verificar que el estilo seguido en el código es limpio y correcto.

6.3. Nodo de control del CDPR

El sistema integrado en ROS 2 tiene como núcleo el nodo *cdpr_controller*, encargado de recibir las posiciones objetivo por las que se quiere desplazar el efector final para así poder aplicarlas en el modelo cinemático desarrollado, con el objetivo de obtener todos los parámetros relacionados con los cables y las poleas del sistema.

El nodo se inicializa y se mantiene activo en todo momento para que pueda recibir todos los mensajes que contienen cada una de las posiciones objetivo, cuyo diseño y funcionalidad se concentran en un único nodo con el objetivo de simplificar la integración y el análisis del sistema completo.

Por ello, se adjunta a continuación un diagrama de clases simplificado que describe los elementos más importantes del nodo *cdpr_controller*, con el objetivo de facilitar la comprensión del mismo, ya que éste se considera el núcleo del sistema.

Figura 6.1: Estructura del nodo `cdpr_controller`

Como se puede observar, la clase principal corresponde al nodo `cdpr_controller`, encargado de gestionar la ejecución del sistema dentro de ROS 2.

Este nodo recibe todas las posiciones o trayectorias objetivo por las que se quiere desplazar el efector final a través de un topic y haciendo uso del modelo cinemático para calcular todos los parámetros asociados a los cables y a las poleas, para posteriormente, publicar toda esta información en distintos topics.

Es por ello que esta organización es la que permite concentrar toda la lógica del sistema en un único nodo siguiendo una estructura que facilita la comprensión del mismo, y donde podrían llevarse a cabo ampliaciones de cara al futuro.

Para una mejor comprensión del comportamiento dinámico del sistema mostrado en la figura, se adjunta un enlace de descarga a un vídeo demostrativo al que se puede acceder haciendo clic aquí.

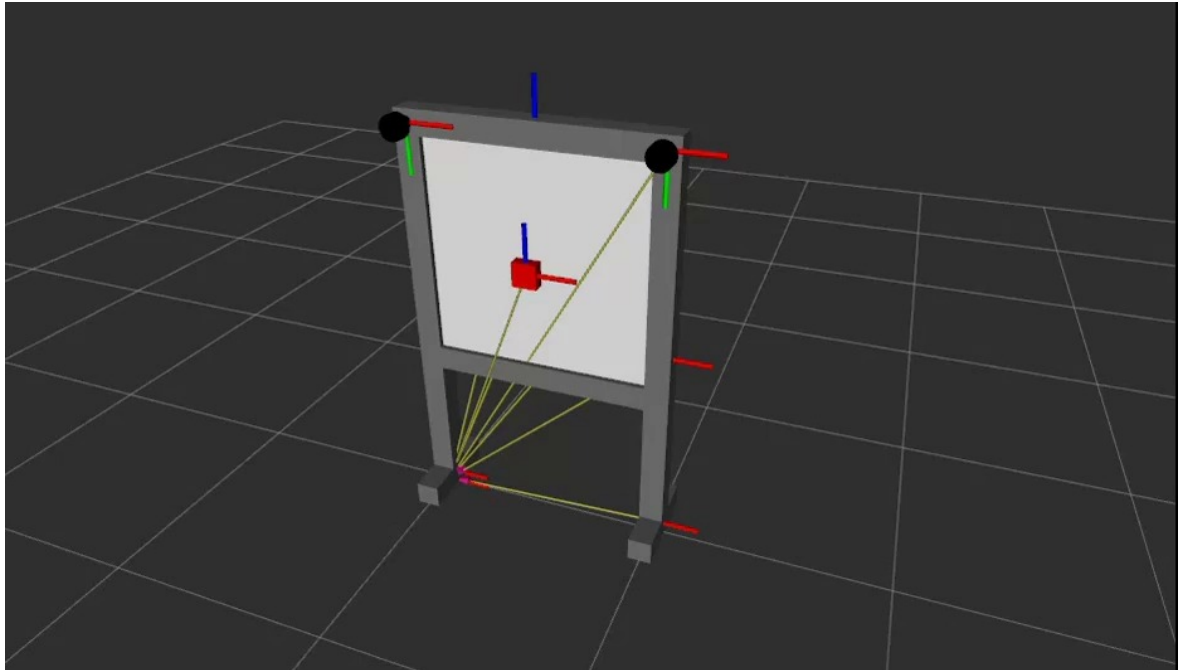


Figura 6.2: Visualización en RViZ durante la ejecución de una trayectoria

6.4. Comunicación mediante topics

La comunicación entre el nodo *cdpr_controller* y el resto del sistema se lleva a cabo utilizando topics que siguen un modelo publicador-suscriptor, lo que permite diferenciar los distintos componentes del sistema y monitorear el estado del mismo en tiempo real.

Por ello, se adjunta a continuación un diagrama de secuencia que muestra el intercambio de mensajes y cómo se produce esta comunicación entre los distintos componentes del sistema.

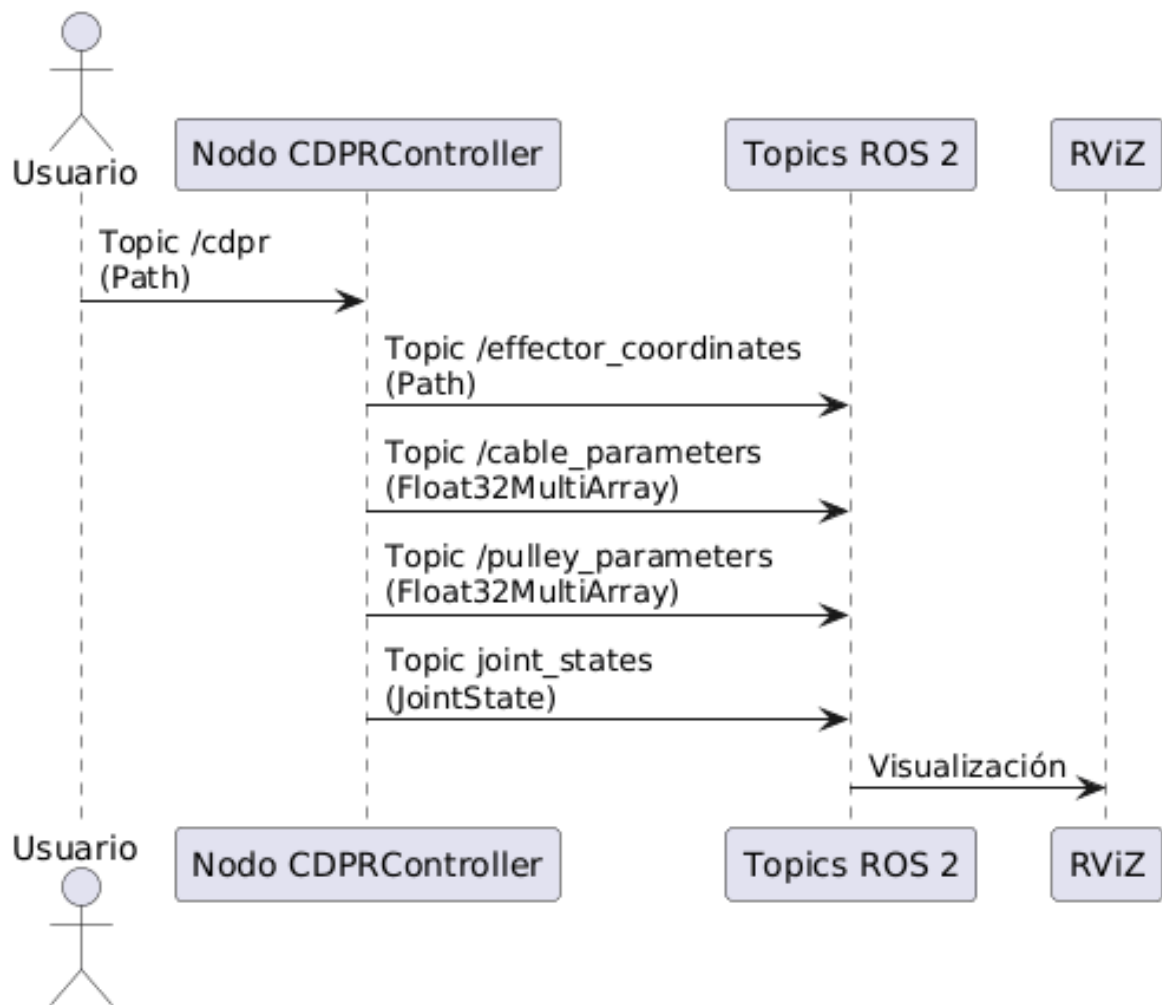


Figura 6.3: Comunicación mediante topics en ROS 2

En primer lugar, el usuario publica una única posición o un conjunto de posiciones objetivo en el topic correspondiente. Después, el nodo *cdpr_controller* recibe este mensaje y va procesando cada una de las posiciones objetivo que ha recibido haciendo uso del modelo cinemático. Y por último, el nodo *cdpr_controller* publica en cada topic correspondiente mensajes sobre la posición del efector final o parámetros asociados a los cables y a las poleas, los cuales son también utilizados por RViZ para poder representar el estado del sistema de forma visual y en tiempo real.

El nodo *cdpr_controller* se suscribe al topic */cdpr*, que recibe mensajes del tipo *nav_msgs/msg/Path*, que contiene una única o una secuencia de posiciones objetivo por las que se quiere desplazar el efector final. Una vez recibida esta información, el nodo publica en distintos topics:

- Topic `/effector_coordinates`: Muestra la posición en cada momento del efector final en el plano XY.
- Topic `/cable_parameters`: Muestra la longitud y el ángulo que forman con la vertical de cada uno de los cables.
- Topic `/pulley_parameters`: Muestra la diferencia de longitudes de cada cable y la diferencia de ángulos de rotación de las poleas entre dos posiciones consecutivas.

Estos topics permiten observar de forma detallada el comportamiento interno del sistema en todo momento

6.5. Gestión de trayectorias y movimientos

Este sistema tiene implementados diferentes modos de comportamiento en función del número de puntos recibidos en el mensaje de tipo `nav_msgs/msg/Path`.

Cuando se recibe una única posición, el sistema interpreta la posición recibida como un posicionamiento fijo del efector final. En caso de que haya recibido dos posiciones, lo que se genera es un desplazamiento entre una posición inicial y una posición final. Y por último, si recibe tres o más posiciones, el sistema lo interpreta como una trayectoria compuesta por todas las posiciones que se le han pasado y por las que el efector final debe pasar en el orden especificado.

Para este último caso, el nodo interpola posiciones intermedias y aplica el modelo cinemático para calcular los parámetros correspondientes en todo momento para cada segmento de la trayectoria. Esto permite simular un movimiento continuo y coherente del efector final dentro del espacio de trabajo.

6.6. Visualización del sistema en RViZ

La integración del sistema y su visualización en RViZ son claves en el desarrollo del mismo, ya que permiten observar de forma mucho más clara el comportamiento del sistema. Para ello, se ha definido un modelo URDF que describe la estructura fija, los cables, las poleas y el efector final.

El lanzamiento conjunto del nodo `cdpr_controller` y del `robot_state_publisher` permiten actualizar en todo momento la visualización del robot en RViZ según se va ejecutando cada trayectoria, cuya configuración facilita la observación de los distintos elementos que componen el sistema y su evolución cuando se encuentran en movimiento.

6.7. Compilación y ejecución del sistema

Para llevar a cabo la ejecución del sistema, se ha definido un procedimiento de instalación basado en colcon, donde el repositorio de Github oficial de este TFG actúa como el directorio *src/* de un workspace, lo que permite compilar y ejecutar el paquete directamente.

Una vez compilado el workspace, la ejecución del sistema completo consiste en la ejecución del nodo *cdpr_controller* y el lanzamiento del entorno de visualización en RViZ. Por otro lado, se publican en el topic correspondiente los mensajes de tipo *nav_msgs/msg/Path* que contienen las posiciones objetivo por las que se desea desplazar el efector final, lo que facilita las pruebas y la validación del comportamiento del sistema.

Este procedimiento asegura la reproducción correcta de todas las pruebas que se lleven a cabo y simplifica la puesta en marcha del sistema en todos los entornos utilizados.

Capítulo 7

Desarrollo hardware

7.1. Introducción

...

7.2. Diseño mecánico del sistema

...

7.3. Sistema electrónico

...

7.4. Programación del microcontrolador

...

7.5. Integración hardware-software

...

7.6. Consideraciones prácticas y limitaciones del prototipo

...

7.7. Resumen

...

Capítulo 8

Pruebas experimentales y validación

8.1. Introducción

...

8.2. Metodología de pruebas

...

8.3. Pruebas de posicionamiento del efector final

...

8.4. Prueba de trayectorias

...

8.5. Comparación entre simulación y resultados

...

8.6. Análisis de errores y fuentes de incertidumbre

...

8.7. Resumen

...

Capítulo 9

Discusión de resultados

9.1. Introducción

...

9.2. Evaluación del cumplimiento de los objetivos

...

9.3. Análisis de los resultados de la simulación

...

9.4. Análisis de los resultados experimentales

...

9.5. Comparación entre comportamiento teórico y real

...

9.6. Limitaciones del sistema desarrollado

...

9.7. Propuesta de mejora y líneas de evolución

...

9.8. Resumen

...

Capítulo 10

Conclusiones y mejoras a futuro

10.1. Introducción

...

10.2. Conclusiones generales

...

10.3. Grado de cumplimiento de los objetivos

...

10.4. Aportaciones del trabajo

...

10.5. Limitaciones finales del sistema

...

10.6. Mejoras a futuro

...

10.7. Cierre del trabajo

...

Bibliografía

- [1] IEEE. Cable-driven parallel robot related research. 2019.
- [2] YouTube. Modelado cinemático - recurso audiovisual 1. <https://www.youtube.com/watch?v=Lgs8SA1bLrc>.
- [3] YouTube. Modelado cinemático - recurso audiovisual 2. <https://www.youtube.com/watch?v=PB4f1p8K2uk>.
- [4] MathWorks. *MATLAB Documentation*.
- [5] Python Software Foundation. *Python 3 Documentation*.
- [6] DevDocs. *C++ Reference Documentation*.
- [7] Open Robotics. *ROS 2 Jazzy Documentation*.
- [8] Open Robotics. *URDF Tutorials*.
- [9] Open Robotics. *RViz Tutorials*.
- [10] Colcon Project. *Colcon Documentation*.
- [11] GitHub. *GitHub Documentation*.
- [12] Open Robotics. *ROS 2 nav_msgs/Path Message Documentation*.
- [13] micro-ros esp-idf component. https://github.com/micro-ROS/micro_ros_esp_idf_component/tree/jazzy.
- [14] eProsima. *Micro XRCE-DDS Documentation*.
- [15] Alberto León. Cable driven parallel robot 2d. https://github.com/aleon2020/cable_driven_parallel_robot_2d.